

**Multi-Metric Text Similarity Evaluation
for Contextual Sentence Reconstruction**

Cole Lamers

University of Wisconsin – Whitewater

Submitted in Partial Fulfillment
of the Requirements for the Degree of
Masters of Science in Computer Science

Advisor:
Dr. Hien Nguyen

08/11/2025

Abstract

This project presents a lightweight text appending and caption evaluation algorithm using lexical and structural similarity metrics such as Jaccard Cosine Dice and ROUGE-L. The approach combines a sliding window technique with fuzzy matching to balance local context with tolerance for transcription noise. This is particularly effective in automatic speech recognition (ASR) outputs. The system evaluates concatenated outputs against the intended output using ROUGE-L scoring as a means of determining the overall accuracy. The algorithm implements a proportional and tiered threshold modifier that allows the text concatenation to be adaptive to: appending overwriting ignoring or detecting false-positives depending on the scores from the threshold values measured. Several test cases were compared using a broad range of threshold modifiers. A real-time real-world test was also conducted using Stephen Colbert's White House Correspondents' Dinner speech due to its length and the variations within the audio processed. This method demonstrates the potential for a real-time string appending process that is fast adaptable accurate and deterministic without the computational overhead of a full-scale LLM.

Introduction

Automatic speech recognition (ASR) systems have become increasingly popular for converting spoken language into text in real time. However the outputs from these systems include fragmented duplicated or misaligned segments that disrupt the flow and readability of the transcription. These issues make it challenging to produce a coherent continuous streams of text especially in live settings like closed captioning or real time transcription services. This project addresses the problem of merging such fragmented outputs into a seamless text stream without relying on LLMs, which are computationally expensive and often impractical for real time use on resource limited devices. Instead the approach combines multiple lexical and structural similarity metrics such as Jaccard Cosine Dice and ROUGE-L with a fuzzy sliding window technique. This enables the system to compare local chunks of text effectively capturing context while tolerating transcription errors and noise.

A key feature of the algorithm is its adaptive tiered threshold system which makes decisions to append, overwrite, ignore, or flag false positives in the text concatenation process. By balancing these concepts, the method will produce more accurate and coherent text streams from ASR outputs. Overall the goal was to create a fast lightweight and deterministic algorithm capable of real time sentence reconstruction improving transcription quality without the resource demands of full scale LLMs. This approach has practical applications in live captioning accessibility and other scenarios requiring efficient reliable speech to text conversion.

Related Work

The challenge of merging streaming ASR outputs into coherent text has been addressed through various strategies throughout the past few decades in academic research. While many early approaches relied on deterministic algorithms and heuristic based rules, LLMs have shifted much of the current focus toward AI driven post processing. This shift has brought accuracy improvements in certain contexts but also introduced significant computational costs, latency, and a reliance on resource intensive infrastructure. Traditional string matching and similarity detection techniques such as Levenshtein distance (Levenshtein, 1966) and the Smith Waterman algorithm (Smith and Waterman, 1981) have long been used in sequence alignment tasks. These methods excelled at identifying insertions and deletions between two strings. However they are computationally intensive when applied repeatedly to real time text streams and do not inherently account for semantic similarity or word order beyond exact character matches. They focused too much on exact precision over probable candidates for similarity.

Recent research has explored lexical similarity metrics such as Jaccard, Cosine, and Dice coefficients for sentence comparison. These metrics treat text as sets of tokens enabling fast comparisons and tolerance for small variations. Studies often apply these metrics individually focusing on a single measure of similarity. While effective in detecting overlaps or duplicates, single metric approaches can be prone to false positives especially when dealing with reordered words reversed sentence structures or linguistically unusual patterns such as poetic

phrasing or “Yoda speak.” ROUGE-L is another metric originally designed for evaluating summarization systems measures the longest common subsequence between two strings capturing both word overlap and order. Some works have adopted ROUGE-L for assessing transcription accuracy but it is less commonly integrated into real time merging pipelines. Instead it is typically used post hoc for evaluation.

Multi-metric strategies have been investigated in limited contexts. Vrbanc & Meštrović (2023) highlight the importance of false positive detection in similarity analysis noting that simple thresholds of a single metric often fail to account for cases where text appears similar but differs in context. This work tended to remain theoretical or was very limited in scope up until now.

The sliding window approach comparing local subsequences of tokens rather than entire sentences has seen use in bioinformatics and plagiarism detection where partial matches are critical. In ASR research, global comparison remains more common potentially missing subtle context shifts in overlapping fragments.

This project builds on the strengths of prior work adopting multiple similarity metrics incorporating sliding window local matching and applying proportional thresholding while addressing the gap between static text evaluation and real time processing. By combining deterministic multi-metric analysis with adaptive decision logic, it aims to be a practical approach toward solving these issues.

Problem Definition

Real time ASR systems produce text in a way that is fundamentally different from static transcription. Rather than generating a complete sentence in one pass they output fragments of text as the audio is processed often revising or expanding those fragments with each subsequent update. This incremental process is prone to significant redundancy and overlapping or duplicate text which must be resolved in order to have a final stream output match exactly what was actually stated.

The transcription output from whisper.cpp illustrates the problem clearly. A fragment might first appear as “YYZ.” As the system hears more of the sentence the next fragment might be “YYZABC.” When concatenated naively, the growing transcript becomes concatenated into “YYZYYZABC” rather than the intended “YYZABC.” The issue is not limited to just simple duplication. The system may also produce partial overlaps where only the last few words differ or generate outputs that appear similar according to basic string matching measures. Examples include sentences where the word order is reversed (“He runs fast” versus “Fast runs he”) or where unusual syntax gives the illusion of similarity despite a semantic difference.

A naive merging strategies like direct string comparison or the application of a single similarity metric fails in these scenarios. Simple equality checks will miss subtle corrections resulting in either lost information or repeated fragments. Basic lexical similarity measures such as Jaccard or Cosine coefficients may return high similarity scores for reordered or restructured sentences leading to false positives. Moreover ASR systems often

make small corrections mid sentence such as replacing the final word which require overwriting rather than discarding the fragment entirely.

These issues are amplified in live transcription environments where both accuracy and latency are critical. While LLMs could in theory infer the correct merged output their use introduces additional problems like building a dedicated machine for that single task, energy costs, reliance on a non-deterministic behavior, processing latencies, and high computational cost. An LLM's output may vary between runs given the same input which is unreliable when in certain contexts, the legalities around accuracy may require live action intervention from a person to make corrections.

The challenge was then to create an algorithm that can process these incoming fragments as they arrive. It should determine whether each fragment represents genuinely new content or a variation of existing content and merge it appropriately. This requires more than just detecting similarities. It also needs the ability to distinguish between overlap check for false positives. The solution must also be computationally lightweight to operate in real time. Not only that, it must be deterministic in the sense that we can predict what it will do with the same two sets of data fed to it. This ensures consistent results able to handle the variability of natural speech without relying on the overhead of LLM based post processing.

Methodology

The algorithm developed for this to work was designed to integrate seamlessly into a streaming ASR using Whisper.cpp by processing each transcription fragment as it arrives and determining in real time whether it should be appended overwrite part of the existing output be ignored or be flagged as a false positive. To achieve this the approach combined multiple similarity metrics with a sliding window comparison strategy guided by a proportional and tiered thresholding system. This combination allows the method to detect meaningful overlap tolerate small transcription errors and avoid the pitfalls of relying on any single similarity measure.

Threshold	VERYLOW	LOW	MEDIUM	HIGH	VERYHIGH
0.05	0.05	0.10	0.15	0.20	0.25
0.06	0.06	0.12	0.18	0.24	0.30
0.07	0.07	0.14	0.21	0.28	0.35
0.08	0.08	0.16	0.24	0.32	0.40
0.09	0.09	0.18	0.27	0.36	0.45
0.1	0.10	0.20	0.30	0.40	0.50
0.11	0.11	0.22	0.33	0.44	0.55
0.12	0.12	0.24	0.36	0.48	0.60
0.13	0.13	0.26	0.39	0.52	0.65

Figure 1: Example of Thresholds

Instead of performing a global comparison between the complete accumulated transcript and the new sentence, the algorithm uses a sliding window approach to focus on local context. Both the current output and the incoming fragment are broken into overlapping windows of fixed size of words in the initial implementation.

Every window in the first text is compared against every window in the second producing a fine grained view of where and how the two fragments align, until a candidate is found (greater than the highest threshold multiplier value, of the current threshold base value).

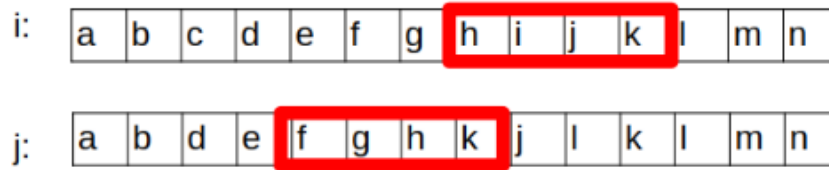


Figure 2: Sliding Window

For each comparison window pair, the algorithm computes three lexical similarity scores: Jaccard, Cosine, and Dice coefficients. These measures are complementary. Jaccard quantifies the proportion of shared unique words, Cosine evaluates the angular similarity between vectorized token frequencies, and Dice emphasizes the harmonic mean of shared elements. By examining the agreement and disagreement between these metrics the algorithm develops a clearer understanding of how closely two text segments are related.

While these three metrics operate across the strings as a whole, ROUGE-L is reserved for full sentence comparisons. ROUGE-L identifies the longest common subsequence between two set of strings. So if Sentence A is “I like to walk my dog” and Sentence B is “I want to walk my big dog”, it will detect the similarity between the two where “I...to walk my...dog” are in a similar contextual order, and base the similarity between the two sentences based on this correct pattern and how proportional each sentence is in terms of length.

The decision making process is determined by a dynamic tiered threshold system. It contains a base value representing the minimum similarity score required for a match to be considered valid. This value is adjusted by multipliers depending on the context and type of match detected. In cases where near exact matches are expected, a higher threshold is applied to reduce the risk of false positives. When it involves more natural variable speech, a lower threshold allows for greater tolerance of line length and variation.

Once similarity scores are calculated and thresholds are applied, the algorithm will choose one of four actions: append, overwrite, ignore, check for false positives if the similarity metrics have high similarity scores, but a very low ROUGE-L score. This tiered logic is designed to maintain both the fluidity of the transcript and the accuracy of its content without introducing unnecessary computational overhead.

General Tests

The first test involved general spoken text containing both partial and full duplicate lines. The speech patterns were intentionally varied to reflect the irregularities of natural language including pauses rephrasing and occasional redundant phrases. This test was a close approximation of real world conditions making it useful for

evaluating the algorithm’s ability to maintain coherent output while removing duplicates and preserving unique information.

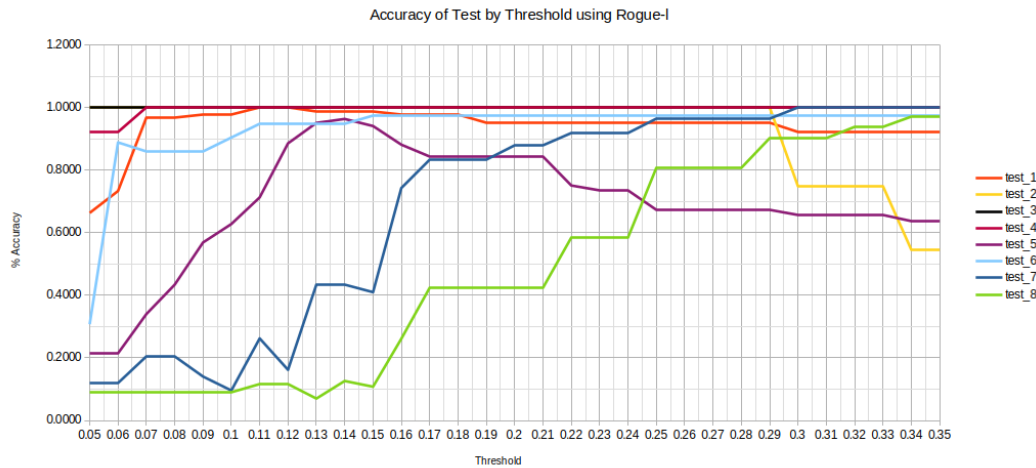


Figure 3: Display of accuracy of each test by threshold used

Subsequent tests targeted more specific conditions. In the second test the fragments differed only in their final few words simulating cases where ASR systems make a minor correction or substitution near the end of a sentence. Here the algorithm was expected to detect the overlap overwrite only the altered portion and preserve the corrected version. The third test contained exact duplicates of sentences challenging the system to recognize these repetitions and avoid unnecessary reappending. The fourth test consisted entirely of unique lines ensuring that the algorithm would append each fragment without misclassifying any as redundant.

The fifth test presented heavily overlapping sentences in which each new fragment contained most of the preceding sentence plus additional content. This was designed to force the algorithm to perform partial appends at precise indexes merging overlapping text while avoiding redundant repetitions. Accuracy in this test depended on the sliding window’s ability to identify the exact point where the overlap ended and the new content began.

Tests six through eight introduced more complex variations intended to provoke false positives. These included sentence reversals where the same words appeared in reverse order, completely reversed sentences, and sequences in which sentence order was flipped from last to first. These cases were particularly challenging because they can yield high similarity scores under basic lexical measures while actually conveying different meaning or structure. These scenarios mimic real speech anomalies such as “Yoda speak” or poetic word ordering requiring the algorithm to be conscientious of false positives.

Stephen Colbert Test

For a real world stress test, the algorithm was leveraged against audio from Stephen Colbert’s White House Correspondents’ Dinner speech. This transcription lasted for twenty four minutes. The choice of material

was intentional because of the humor, pauses for laughter, a skit was in the video, and a couple different people spoke during it, which could cause some possible issues during the concatenation process. This was to mimic real world scenarios as much as possible, and because of how articular Stephen Colbert is at speaking, it made for the audio coming through to be precisely worded so we did not have to worry about slang or a very casual speaking tone.

The speech was processed in real time using `whisper.cpp` which produced a continuous stream of sentences as text strings. The algorithm was applied to this stream with the same sliding window and multi-metric similarity framework used in the earlier tests. Its task was to produce a clean continuous transcript that matched the total concatenated output from the YouTube subtitle file as much as possible. This subtitle file provided a great reference to measure the final concatenated output and detect the accuracy appropriately.

Two primary comparisons were performed. First the algorithm's output was compared against the `whisper.cpp` transcription. Second it was compared against the YouTube subtitle (un-concatenated) using the ROUGE-L metric to measure sequential alignment of words and phrases. By comparing the two, we could see how much of the concatenation issues stem from actually processing the algorithm against text versus processing it against a continuous, stochastic input-output of audio to text with high degrees of variability.

Overall the algorithm succeeded in producing a coherent transcript that preserved the meaning of the original speech while significantly reducing duplication and redundancy compared to the unprocessed ASR output. This confirmed its ability to function effectively in a live transcription environment where language is far from predictable and it highlighted the importance of how threshold values affect the ability for the algorithm to determine the proper task to perform.

Results

The evaluation of the proposed algorithm across both types of testing scenarios produced a clear picture of its strengths. The controlled tests were created to demonstrate specific transcription challenges that would be effective in identifying overlaps eliminating redundancies and avoiding false positives. The real world Stephen Colbert stress test reinforced these findings while also revealing the impact of threshold tuning on performance in more variable linguistic environments. In the general tests the algorithm consistently achieved high ROUGE-L scores when the thresholds when threshold values were higher, due to the strict requirement for them to ensure they guaranteed, found a match. Tests 5 through 8 were intentionally constructed to provoke false positives. In these cases, the algorithm benefited from stricter similarity requirements which prevented misleading matches from being merged incorrectly. The adaptive nature of the tiered threshold system allowed for this flexibility but the results suggested that no single threshold configuration was optimal for all scenarios.

However for the Colbert Tests and Test 1, lower threshold values were faring much better because they needed more flexibility in determining certain probabilities for candidates as opposed to being more strict. In

essence, the strictness was a hindrance to accuracy when performing lexical string comparisons in real world examples for similarity between the two. The results revealed an interesting pattern consistent with earlier controlled tests. Lower threshold settings tended to produce higher accuracy scores in this real world scenario as they allowed the algorithm to tolerate minor differences between fragments without discarding potentially important content. In contrast higher thresholds while effective at filtering false positives in the synthetic tests sometimes rejected legitimate overlaps in the Colbert speech especially in moments of rapid wordplay or subtle rephrasing. When comparing the Colbert test against the subtitle file from YouTube, the algorithm achieved very high accuracy scores that were consistent with the best performing threshold configurations observed in the controlled tests.

Overall the results indicated that the algorithm is well suited to a range of transcription contexts. The unique aspect of this algorithm is that it offers the ability to fine tune it to the need of the work being done maximizing performance. Lower thresholds favor natural variable speech where flexibility is important to avoid content loss, while higher thresholds excelled in more structured environments where false positives were much more likely to come about (like in poetry for instance). This design helps to ensure that these results can be achieved without the latency and resource demands associated with LLM based solutions making the method viable for real time captioning broadcast transcription and accessibility applications.

Discussion

The results of this project highlight both the promise and challenges of applying a lightweight multi-metric similarity approach to real time text concatenation for automatic speech recognition outputs. The algorithm successfully addresses several common pitfalls described from the current literature explaining this today within streaming transcription. This is an important contribution given the growing demand for efficient scalable speech to text processing in resource constrained or latency sensitive environments where an LLM might not be the proper tool for all problems. The algorithm does have some minor issues that do need to be stated. The current sliding window approach has a theoretical lower bound of $O(n*m)$ complexity, which may face scalability challenges with very long sentences or highly repetitive text streams. In practice, however, it has remained performant for most transcription lengths. Additionally the necessity of manually tuning thresholds for different scenarios suggests updates that could be made to make it easier to fine tune without adjusting the code. Overall this can really help with the work related to string concatenating issues that exist with current research. Also, from a practical point of view, the algorithm is well positioned for applications such as live closed captioning where minimizing latency and computational load is critical. Its deterministic nature provides predictable outputs which is valuable in accessibility contexts where accuracy and consistency directly impact user experience. Furthermore it offers potential to enhance semi-automated transcription workflows reducing the manual burden on human transcribers by pre-cleaning ASR outputs.

Future Work

While the current algorithm displays the start to promising work related to real time text concatenation, there still are several additional areas for improvement with this. One important direction is the development of adaptive threshold tuning. Currently threshold values must be manually set and fine tuned depending on the use case and dataset. An automated or context aware thresholding mechanism could dynamically adjust sensitivity based on the input's complexity or length from the ASR system which could help with improving accuracy and induce proper false positive detection when it is needed.

Another potential enhancement could be with optimizing the sliding window comparison to better handle very long sentences or streams. Although the current $O(n*m)$ complexity is acceptable in all the tests that were conducted, the scalability of it is something to be conscientious of. A divide and conquer windowing approach could be considered to facilitate a faster means of making comparisons instead of just comparing two strings in their entirety numerous times. The similarity metrics used in evaluation could also be improved in some other ways as well. The current system set up does not take into account the whole sentence as a metric for comparison. That could be an indicator check from the start to see if it's even worth parsing, thus saving time on running the sliding window part and drastically reducing processing.

Finally conducting more extensive real world testing with diverse speakers languages and noisy environments would validate the algorithm's generalization and reveal practical challenges that might arise outside controlled test cases. Running against other languages and types of audio could help with building this up further to increase it's accuracy and better facilitate a more predictable algorithm. These future directions aim to make the system more adaptive, efficient, and semantically aware by enhancing its utility in real time speech to text applications.

Conclusion

This project presents a lightweight multi metric algorithm for real time text concatenation that effectively merges fragmented transcription outputs without relying on large language models. By combining lexical and structural similarity measures with a fuzzy sliding window and dynamic thresholding the system accurately appends overwrites or ignores text fragments while handling transcription noise and false positives. Testing on varied datasets including a lengthy real world speech shows the algorithm balances accuracy and efficiency making it well suited for live captioning and streaming transcription applications where speed and resource constraints matter. While it is not aiming to replace LLM's fully, this approach offers a practical, lightweight solution to improve the readability and coherence of automatic speech recognition outputs in real time.

Bibliography and References

- Abdallah, A., Piryani, B., & Jatowt, A. (2023). Exploring the state of the art in legal QA systems. *Journal of Big Data*, 10(1), 127.
- Celikten, T., & Onan, A. (2025). Exploring text similarity in human and AI-generated scientific abstracts: A comprehensive analysis. *IEEE Access*.
- Clark, A., Fox, C., & Lappin, S. (Eds.). (2013). *The handbook of computational linguistics and natural language processing*. John Wiley & Sons.
- Cozzolino, I. (2023). Fuzzy spectral clustering methods for textual data.
- Ghafoor, M., & Akutsu, T. (2024). On the generative power of ReLU network for generating similar strings. *IEEE Access*, 12, 52603–52622.
- Grinsztajn, L., Oyallon, E., Kim, M. J., & Varoquaux, G. (2023). Vectorizing string entries for data processing on tables: When are larger language models better? *arXiv preprint arXiv:2312.09634*.
- Gupta, A., & Goyal, R. (2024). A generative AI-driven method-level semantic clone detection based on the structural and semantical comparison of methods. *IEEE Access*, 12, 70773–70791.
- Hassanzadeh, O., Sadoghi, M., & Miller, R. J. (2007, September). Accuracy of approximate string joins using grams. In *QDB* (pp. 11–18).
- Henderi, H., & Winarno, W. (2021). Text mining an automatic short answer grading (ASAG): Comparison of three methods of cosine similarity, Jaccard similarity, and Dice's coefficient. *Journal of Applied Data Sciences*, 2(2).
- Hussein, M. M. A. (2025). LLM-enhanced entity matching: Comparative analysis of traditional and modern techniques (Master's thesis, ETH Zurich).
- Ji, S., Li, G., Li, C., & Feng, J. (2009, April). Efficient interactive fuzzy keyword search. In *Proceedings of the 18th international conference on World wide web* (pp. 371–380).
- Jiang, Y., Li, G., Feng, J., & Li, W. S. (2014). String similarity joins: An experimental evaluation. *Proceedings of the VLDB Endowment*, 7(8), 625–636.
- Kimmet, B., Thoma, A., & Srinivasan, V. (2016, July). Fuzzy joins in MapReduce: Edit and Jaccard distance. In *2016 7th International Conference on Information, Intelligence, Systems & Applications (IISA)* (pp. 1–6). IEEE.
- Lee, G. G., Seo, J., Lee, S., Jung, H., Cho, B. H., Lee, C., ... & Kim, K. (2001, November). SiteQ: Engineering high performance QA system using lexico-semantic pattern matching and shallow NLP. In *TREC*.
- Mehri, B., Goussard, Y., & Trépanier, M. (2023). Lookup table string similarity algorithm. *Bureau de Montreal, Université de Montreal*.
- Peeters, R., Steiner, A., & Bizer, C. (2023). Entity matching using large language models. *arXiv preprint arXiv:2310.11244*.

- Pradeep, R., Hui, K., Gupta, J., Lelkes, A. D., Zhuang, H., Lin, J., ... & Tran, V. Q. (2023). How does generative retrieval scale to millions of passages? arXiv preprint arXiv:2305.11841.
- Rogers, A., Gardner, M., & Augenstein, I. (2023). QA dataset explosion: A taxonomy of NLP resources for question answering and reading comprehension. *ACM Computing Surveys*, 55(10), 1–45.
- Smith, T. F., & Waterman, M. S. (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1), 195–197.
- Tao, W. (2018). Approximate string joins with abbreviations (Doctoral dissertation, Massachusetts Institute of Technology).
- Vrbanec, T., & Meštrović, A. (2023). Comparison study of unsupervised paraphrase detection: Deep learning the key for semantic similarity detection. *Expert Systems*, 40(9), e13386.
- Wandelt, S., Deng, D., Gerdjikov, S., Mishra, S., Mitankin, P., Patil, M., ... & Leser, U. (2014). State-of-the-art in string similarity search and join. *ACM Sigmod Record*, 43(1), 64–76.
- Wang, J., Lin, C., Li, M., & Zaniolo, C. (2019). An efficient sliding window approach for approximate entity extraction with synonyms. In *EDBT* (pp. 109–120).
- Xiao, C., Wang, W., Lin, X., Yu, J. X., & Wang, G. (2011). Efficient similarity joins for near-duplicate detection. *ACM Transactions on Database Systems (TODS)*, 36(3), 1–41.
- Xiao, G., Wang, J., Lin, C., & Zaniolo, C. (2022, May). Highly efficient string similarity search and join over compressed indexes. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)* (pp. 232–244). IEEE.